

Relaxing some `constexpr` restrictions

Document #: P2448R0
Date: 2021-10-13
Project: Programming Language C++
Audience: CWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Abstract](#)
- [2 Revision History](#)
- [3 Maybe Not Now, But Soon](#)
- [4 Sometimes Maybe, Sometimes Always](#)
- [5 Onwards to `constexpr` classes](#)
- [6 Proposal](#)
- [7 References](#)

§ 1 Abstract

There are two rules about `constexpr` programming that make code ill-formed or ill-formed (no diagnostic required) when functions or function templates are marked `constexpr` that might never evaluate to a constant expression. But... so what if they don't? The goal of this paper is to stop diagnosing problems that don't exist.

§ 2 Revision History

A draft of the first revision of this paper was discussed in an [EWG telecon](#), where the following poll was taken:

send P2448 to electronic polling, targeting CWG for C++23.

SF	F	N	A	SA
10	8	1	0	0

This first published revision thus targets CWG.

§ 3 Maybe Not Now, But Soon

`constexpr` functions and function templates in C++ generally speaking mean *maybe* `constexpr`. Not all instantiations or evaluations must be invocable at compile time, it's just that there must be at least one set of function arguments in at least one instantiation that works.

And this isn't just generally speaking, this is enshrined as a rule: 9.2.6 [\[dcl.constexpr\]/6](#):

For a `constexpr` function or `constexpr` constructor that is neither defaulted nor a template, if no argument values exist such that an invocation of the function or constructor could be an evaluated subexpression of a core constant expression, or, for a constructor, an evaluated subexpression of the initialization full-expression of some constant-initialized object ([basic.start.static]), the program is ill-formed, no diagnostic required.

Here is an example of a program that violates this rule:

```
void f(int& i) {
    i = 0;
}

constexpr void g(int& i) {
    f(i);
}
```

`g` unconditionally calls `f`, which is not a `constexpr` function, so there does not exist any invocation that would be a constant expression. Ill-formed, no diagnostic required. `gcc` and `msvc` both diagnose this.

Now, one could argue that this diagnosis is a good thing: that `constexpr` annotation on `g` makes no sense! It can't be a constant expression, so having that specifier on the function is misleading. Diagnosing this error helps the programmer realize that they were mistaken and they can take steps to change this. That seems compelling enough.

Consider instead this example:

```
#include <optional>

constexpr void h(std::optional<int>& o) {
    o.reset();
}
```

Here, `h` is the exact same kind of function as `g`: we have a function unconditionally calling another non-`constexpr` function. At least, that's true for C++17. It won't be true in C++23, and the answer for C++20 depends on how vendors choose to implement [\[P2231R1\]](#). Here the answer depends: some functions can easily be `constexpr` in one standard but not in earlier ones.

Now, the sanctioned way to fix this code is to write:

```
#include <optional>

#if __cpp_lib_optional >= 202106
constexpr
#endif
```

```
void h(std::optional<int>& o) {  
    o.reset();  
}
```

This way, `h` will be `constexpr` when the appropriate library changes are made, allowing it to be. But... is this better?

I would argue it's not. The language rules for `constexpr` have expanded in every language standard since `constexpr` was introduced. Standard library support will always lag that. Third-party library support likely even more so. So the answer to the question "are there any arguments for which this function can be a constant expression?" can easily be No in C++N but Yes in C++N+1, for a wide variety of functions. Does forcing conscientious library authors to take painstaking care in conditionally marking function `constexpr` provide value to the ecosystem? I'm skeptical that it does.

Moreover, while it's possible to write the above for `std::optional`, I'm not sure that it's common for other library to provide macros that can be used to mark functions conditionally `constexpr` like this. Or, indeed, if there is even another such example. So if I'm a consumer of a library that might have some functionality `constexpr` in one version but more functionality `constexpr` in the next, I always have to lag.

Such diagnosis may have made sense in the C++11 days, but now that we're approaching C++23 where more and more things are `constexpr` and more and more libraries will mark more of their functions `constexpr` because they can be, it seems strictly better to just reserve diagnosing `constexpr` violations to the place where we already have to diagnose them: when you write code that *must* be evaluated at compile time.

Put differently, the current rule is there must be some tuple (function arguments, template arguments) for which a `constexpr` function invocation is a constant expression. But there's really another input here: (function arguments, template arguments, version). The version here might be the language version, it might be a bunch of library versions. Ultimately, it's a question of time. A function may not be `constexpr` today, but it may be `constexpr` soon. Diagnosing it as not being `constexpr` yet seems harmful to the question of evolving code.

§ 4 Sometimes Maybe, Sometimes Always

`constexpr` usually means *maybe* `constexpr`. But sometimes it actually means *always* `constexpr`. That case comes up with explicitly defaulted functions. From 9.5.2 [\[dcl.fct.def.default\]/3](#):

An explicitly-defaulted function that is not defined as deleted may be declared `constexpr` or `constexpr` only if it is `constexpr`-compatible ([special], [class.compare.default]). A function explicitly defaulted on its first declaration is implicitly inline ([dcl.inline]), and is implicitly `constexpr` ([dcl.constexpr]) if it is `constexpr`-compatible.

Let's say I'm writing a wrapper class template that I'm intending to be usable during compile time:

```
template <typename T>  
struct Wrapper {
```

```
constexpr Wrapper() = default;
constexpr Wrapper(Wrapper const&) = default;
constexpr Wrapper(T const& t) : t(t) { }

constexpr T get() const { return t; }
constexpr bool operator==(Wrapper const&) const = default;
private:
    T t;
};
```

I might take the strategy of just marking every function `constexpr`. Both for consistency and also as a strategy to avoid forgetting to mark some functions `constexpr`.

But then I try to use it:

```
struct X {
    X();
    bool operator==(X const&) const;
};

Wrapper<X> x;
```

None of this code is trying to evaluate anything during constant evaluation, yet it is already ill-formed. gcc and clang already diagnose at this point. msvc and icc do not, but this rule is a mandatory diagnostic, so they are mistaken. Although, even here, none of the compilers care that I erroneously marked the default constructor `constexpr`. And none of them care that I erroneously marked the copy constructor `constexpr`. gcc and clang are only diagnosing `operator==` here (even though I'm not even using it).

But why do we even have this rule in the first place? Note that if I wrote my equality operator this way:

```
constexpr bool operator==(Wrapper const& rhs) const {
    return t == rhs.t;
}
```

That is, manually writing out what the defaulted version does, even with marking it `constexpr`, this code is perfectly valid C++ code. It doesn't matter that I'm marking this function `constexpr`, because in this context, the only requirement is that *some* instantiation can be a constant expression. But because I want to `= default` it, suddenly *every* instantiation has to be a constant expression?

What this means is that the correct way to write my wrapper type is:

```
template <typename T>
struct Wrapper {
    Wrapper() = default;
    Wrapper(Wrapper const&) = default;
    constexpr Wrapper(T const& t) : t(t) { }

    constexpr T get() const { return t; }
    bool operator==(Wrapper const&) const = default;
private:
    T t;
};
```

So I have some functions marked `constexpr` and some not, but all of them are still usable during constant evaluation time (where appropriate based on τ). The lack of consistency here is a bit jarring. `constexpr` functions should always be *maybe* `constexpr`, not *must be* `constexpr`.

§ 5 Onwards to `constexpr` classes

We have a proposal in front of us to allow annotating the entire class as being `constexpr`, to avoid all these extra annotations [P2350R1]. And that proposal currently runs into both issues:

```
template <typename T>
struct Wrapper constexpr {
    Wrapper() = default;
    Wrapper(Wrapper const&) = default;
    Wrapper(T const& t) : t(t) { }

    void reset() { t.reset(); }
    bool operator==(Wrapper const&) const = default;
private:
    std::optional<T> t;
};
```

Are all of these functions okay? I would argue that they *should* all be okay, but per the wording they're currently not.

I want `Wrapper` to be entirely `constexpr` where feasible. Some of those functions may not be `constexpr` for all types, and that's fine. Some of these functions may not be able to be `constexpr` in C++N but may be later, and I don't want to have to go back and either annotate against this (which I don't think P2350 even allows room for) or stop using this feature and go back to manually marking and even more annotations.

§ 6 Proposal

Strike 9.2.6 [dcl.constexpr]/6 and the example following it, in its entirety, along with the second sentence in /7:

- ~~6 For a constexpr function or constexpr constructor that is neither defaulted nor a template, if no argument values exist such that an invocation of the function or constructor could be an evaluated subexpression of a core constant expression, or, for a constructor, an evaluated subexpression of the initialization full-expression of some constant-initialized object ([basic.start.static]), the program is ill-formed, no diagnostic required.~~

~~{Example 4:~~

```
constexpr int f(bool b)
{ return b ? throw 0 : 0; } // OK
constexpr int f() { return f(true); } // ill formed, no diagnostic
    required

struct B {
```

```

constexpr B(int x) : i(0) { } // x is unused
int i;
};

int global;

struct D : B {
constexpr D() : B(global) { } // ill formed, no diagnostic
required
// lvalue to rvalue conversion on
non-constant global
};

— end example}

```

- 7 If the instantiated template specialization of a constexpr function template or member function of a class template would fail to satisfy the requirements for a constexpr function, that specialization is still a constexpr function, even though a call to such a function cannot appear in a constant expression. ~~If no specialization of the template would satisfy the requirements for a constexpr function when considered as a non-template function, the template is ill-formed, no diagnostic required.~~

Strike part of 9.5.2 [\[dcl.fct.def.default\]/3](#) and fix the example (which is already wrong at the moment, since default-initializing an `int` during constant evaluation is ok):

- 3 An explicitly-defaulted function that is not defined as deleted may be declared `constexpr` or `constexpr` ~~only if it is constexpr-compatible ([special], [class.compare.default])~~. A function explicitly defaulted on its first declaration is implicitly inline ([`dcl.inline`]), and is implicitly `constexpr` ([`dcl constexpr`]) if it is constexpr-compatible.
- 4 [Example 1:

```

struct S {
-   constexpr S() = default; // error: implicit S() is not
    constexpr
    S(int a = 0) = default; // error: default argument
    void operator=(const S&) = default; // error: non-matching return type
    ~S() noexcept(false) = default; // OK, despite mismatched
        exception specification
private:
    int i;
    S(S&); // OK: private copy constructor
};
S::S(S&) = default; // OK: defines copy constructor

struct T {
    T();
    T(T &&) noexcept(false);
};
struct U {
    T t;
    U();
    U(U &&) noexcept = default;
};

```

```
};  
U u1;  
U u2 = static_cast<U&&>(u1);           // OK, calls std::terminate if T::  
    T(T&&) throws
```

— *end example*]

§ 7 References

[P2231R1] Barry Revzin. 2021-02-12. Add further constexpr support for optional/variant.

<https://wg21.link/p2231r1>

[P2350R1] Andreas Fertig. 2021-07-15. constexpr class.

<https://wg21.link/p2350r1>